EXEMPLOS DE CÓDIGO - FACTORY PATTERNS COM DI

Guia de Execução e Código Fonte Completo

INSTRUÇÕES GERAIS

Todos os projetos usam .NET 8 e podem ser executados com:

```
bash

dotnet run --project <caminho-do-projeto>
```

Estrutura de diretórios esperada:

```
exemplos/
├── 01-simple-factory/
├── 02-factory-method/
├── 03-abstract-factory/
├── 04-di-integration/
└── 05-project-factory/
```

EXEMPLO 1: SIMPLE FACTORY (20 minutos)

Objetivo

Demonstrar Simple Factory básico e suas limitações antes de integrar com DI.

Arquivos do Projeto

SimpleFactory.csproj

```
xml
```

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.DependencyInjection" Version="8.0.0" />
  </ItemGroup>
</Project>
```

Program.cs

csharp

```
using Microsoft.Extensions.DependencyInjection;

// ===== INTERFACES =====
public interface INotifier
{
    void Send(string recipient, string message);
}

// ===== IMPLEMENTAÇÕES =====
public class EmailNotifier : INotifier
{
    public void Send(string recipient, string message)
    {
        Console.WriteLine($"[EMAIL] Para: {recipient}");
        Console.WriteLine($"[EMAIL] Mensagem: {message}");
    }
}

public class SmsNotifier : INotifier
{
    public void Send(string recipient, string message)
    {
        Console.WriteLine($"[SMS] Para: {recipient}");
        Console.WriteLine($"[SMS] Mensagem: {message}");
    }
}

// ===== SIMPLE FACTORY (SEM DI) =====
public static class NotifierFactory
{
    // Problema: Instanciação direta = acoplamento forte
    public static INotifier Create(string type)
    {
        return type.ToLower() switch
        {
            "email" => new EmailNotifier(),
            "sms" => new SmsNotifier(),
            _ => throw new ArgumentException($"Tipo desconhecido: {type}")
        };
    }
}

// ===== VERSÃO MELHORADA COM DI =====
public interface INotifierFactory
{
    INotifier Create(string type);
```

```
}

public class NotifierFactoryWithDI : INotifierFactory
{
    private readonly IServiceProvider _serviceProvider;

    public NotifierFactoryWithDI(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public INotifier Create(string type)
    {
        return type.ToLower() switch
        {
            "email" => _serviceProvider.GetRequiredService<EmailNotifier>(),
            "sms" => _serviceProvider.GetRequiredService<SmsNotifier>(),
            _ => throw new ArgumentException($"Tipo desconhecido: {type}")
        };
    }
}

// ===== PROGRAMA PRINCIPAL =====
class Program
{
    static void Main()
    {
        Console.WriteLine("=== EXEMPLO 1: SIMPLE FACTORY ===\n");

        // Parte 1: Simple Factory tradicional
        Console.WriteLine("--- Sem DI (Simple Factory tradicional) ---");
        var emailNotifier = NotifierFactory.Create("email");
        emailNotifier.Send("user@example.com", "Pedido confirmado!");

        var smsNotifier = NotifierFactory.Create("sms");
        smsNotifier.Send("+5511999999999", "Seu código: 1234");

        Console.WriteLine("\n--- Com DI (Factory melhorado) ---");

        // Configurar DI
        var services = new ServiceCollection();
        services.AddTransient<EmailNotifier>();
        services.AddTransient<SmsNotifier>();
        services.AddSingleton<INotifierFactory, NotifierFactoryWithDI>();

        var provider = services.BuildServiceProvider();
        var factory = provider.GetRequiredService<INotifierFactory>();
```

```
        // Usar factory
        var email = factory.Create("email");
        email.Send("admin@example.com", "Sistema iniciado");

        var sms = factory.Create("sms");
        sms.Send("+5511888888888", "Alerta de segurança");

        Console.WriteLine("\n=== FIM DO EXEMPLO 1 ===");
    }
}
```

EXEMPLO 2: FACTORY METHOD (30 minutos)

Objetivo

Demonstrar Factory Method pattern com template method e extensibilidade via herança.

NotificationService.csproj

xml

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.DependencyInjection" Version="8.0.0" />
  </ItemGroup>
</Project>
```

Program.cs

csharp

```
using Microsoft.Extensions.DependencyInjection;

// ===== PRODUTOS =====
public interface INotifier
{
    void Send(string recipient, string message);
}

public class EmailNotifier : INotifier
{
    public void Send(string recipient, string message)
    {
        Console.WriteLine($"📧 Email para {recipient}: {message}");
    }
}

public class SmsNotifier : INotifier
{
    public void Send(string recipient, string message)
    {
        Console.WriteLine($"📱 SMS para {recipient}: {message}");
    }
}

public class PushNotifier : INotifier
{
    public void Send(string recipient, string message)
    {
        Console.WriteLine($"🔔 Push para {recipient}: {message}");
    }
}

// ===== CREATOR ABSTRATO COM FACTORY METHOD =====
public abstract class NotificationService
{
    // Factory Method - subclasses implementam
    protected abstract INotifier CreateNotifier();

    // Template Method - usa o Factory Method
    public void NotifyCustomer(string recipient, string message)
    {
        Console.WriteLine($"[{GetType().Name}] Preparando notificação...");

        var notifier = CreateNotifier();
        notifier.Send(recipient, message);
```

```
        Console.WriteLine($"[{GetType().Name}] Notificação enviada com sucesso!\n");
    }
}

// ===== CREATORS CONCRETOS =====
public class EmailNotificationService : NotificationService
{
    private readonly EmailNotifier _emailNotifier;

    public EmailNotificationService(EmailNotifier emailNotifier)
    {
        _emailNotifier = emailNotifier;
    }

    protected override INotifier CreateNotifier()
    {
        return _emailNotifier;
    }
}

public class SmsNotificationService : NotificationService
{
    private readonly SmsNotifier _smsNotifier;

    public SmsNotificationService(SmsNotifier smsNotifier)
    {
        _smsNotifier = smsNotifier;
    }

    protected override INotifier CreateNotifier()
    {
        return _smsNotifier;
    }
}

public class PushNotificationService : NotificationService
{
    private readonly PushNotifier _pushNotifier;

    public PushNotificationService(PushNotifier pushNotifier)
    {
        _pushNotifier = pushNotifier;
    }

    protected override INotifier CreateNotifier()
    {
        return _pushNotifier;
```

```
    }
}

// ===== PROGRAMA =====
class Program
{
    static void Main()
    {
        Console.WriteLine("=== EXEMPLO 2: FACTORY METHOD ===\n");

        // Configurar DI
        var services = new ServiceCollection();
        services.AddTransient<EmailNotifier>();
        services.AddTransient<SmsNotifier>();
        services.AddTransient<PushNotifier>();
        services.AddTransient<EmailNotificationService>();
        services.AddTransient<SmsNotificationService>();
        services.AddTransient<PushNotificationService>();

        var provider = services.BuildServiceProvider();

        // Usar diferentes implementações via polimorfismo
        NotificationService service;

        service = provider.GetRequiredService<EmailNotificationService>();
        service.NotifyCustomer("cliente@example.com", "Seu pedido foi aprovado!");

        service = provider.GetRequiredService<SmsNotificationService>();
        service.NotifyCustomer("+5511999999999", "Código de verificação: 5678");

        service = provider.GetRequiredService<PushNotificationService>();
        service.NotifyCustomer("user123", "Nova mensagem recebida");

        Console.WriteLine("=== FIM DO EXEMPLO 2 ===");
    }
}
```

EXEMPLO 3: ABSTRACT FACTORY (40 minutos)

Objetivo

Demonstrar Abstract Factory para criar famílias de objetos relacionados (sistema de relatórios).

ReportSystem.csproj

xml

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.DependencyInjection" Version="8.0.0" />
  </ItemGroup>
</Project>
```

Program.cs

csharp

```
using Microsoft.Extensions.DependencyInjection;

// ===== PRODUTOS ABSTRATOS =====
public interface IReportHeader
{
    void Render(string title);
}

public interface IReportTable
{
    void Render(string[][] data);
}

public interface IReportChart
{
    void Render(double[] values);
}

// ===== FAMÍLIA PDF =====
public class PdfReportHeader : IReportHeader
{
    public void Render(string title)
    {
        Console.WriteLine($"[PDF] ==== {title.ToUpper()} ====");
    }
}

public class PdfReportTable : IReportTable
{
    public void Render(string[][] data)
    {
        Console.WriteLine("[PDF] Tabela:");
        foreach (var row in data)
        {
            Console.WriteLine($"[PDF]   | {string.Join(" | ", row)} |");
        }
    }
}

public class PdfReportChart : IReportChart
{
    public void Render(double[] values)
    {
        Console.WriteLine("[PDF] Gráfico de barras:");
        for (int i = 0; i < values.Length; i++)
        {
```

```
            Console.WriteLine($"[PDF]   Mês {i + 1}: {new string('', (int)values[i])} {values[i]}");
        }
    }
}

// ===== FAMÍLIA EXCEL =====
public class ExcelReportHeader : IReportHeader
{
    public void Render(string title)
    {
        Console.WriteLine($"[Excel] === {title} ===");
    }
}

public class ExcelReportTable : IReportTable
{
    public void Render(string[][] data)
    {
        Console.WriteLine("[Excel] Planilha:");
        for (int i = 0; i < data.Length; i++)
        {
            Console.WriteLine($"[Excel]   Linha {i + 1}: {string.Join(", ", data[i])}");
        }
    }
}

public class ExcelReportChart : IReportChart
{
    public void Render(double[] values)
    {
        Console.WriteLine("[Excel] Gráfico de linhas:");
        Console.WriteLine($"[Excel]   Valores: {string.Join(" -> ", values)}");
    }
}

// ===== FAMÍLIA HTML =====
public class HtmlReportHeader : IReportHeader
{
    public void Render(string title)
    {
        Console.WriteLine($"[HTML] <h1>{title}</h1>");
    }
}

public class HtmlReportTable : IReportTable
{
    public void Render(string[][] data)
```

```
    {
        Console.WriteLine("[HTML] <table>");
        foreach (var row in data)
        {
            Console.WriteLine($"[HTML]   <tr><td>{string.Join("</td><td>", row)}</td></tr>");
        }
        Console.WriteLine("[HTML] </table>");
    }
}

public class HtmlReportChart : IReportChart
{
    public void Render(double[] values)
    {
        Console.WriteLine("[HTML] <canvas id='chart'>");
        Console.WriteLine($"[HTML]   data: [{string.Join(", ", values)}]");
        Console.WriteLine("[HTML] </canvas>");
    }
}

// ===== ABSTRACT FACTORY =====
public interface IReportFactory
{
    IReportHeader CreateHeader();
    IReportTable CreateTable();
    IReportChart CreateChart();
}

// ===== CONCRETE FACTORIES =====
public class PdfReportFactory : IReportFactory
{
    private readonly IServiceProvider _serviceProvider;

    public PdfReportFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IReportHeader CreateHeader() => _serviceProvider.GetRequiredService<PdfReportHeader>();
    public IReportTable CreateTable() => _serviceProvider.GetRequiredService<PdfReportTable>();
    public IReportChart CreateChart() => _serviceProvider.GetRequiredService<PdfReportChart>();
}

public class ExcelReportFactory : IReportFactory
{
    private readonly IServiceProvider _serviceProvider;
```

```
    public ExcelReportFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IReportHeader CreateHeader() => _serviceProvider.GetRequiredService<ExcelReportHeader>();
    public IReportTable CreateTable() => _serviceProvider.GetRequiredService<ExcelReportTable>();
    public IReportChart CreateChart() => _serviceProvider.GetRequiredService<ExcelReportChart>();
}

public class HtmlReportFactory : IReportFactory
{
    private readonly IServiceProvider _serviceProvider;

    public HtmlReportFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IReportHeader CreateHeader() => _serviceProvider.GetRequiredService<HtmlReportHeader>();
    public IReportTable CreateTable() => _serviceProvider.GetRequiredService<HtmlReportTable>();
    public IReportChart CreateChart() => _serviceProvider.GetRequiredService<HtmlReportChart>();
}

// ===== GERADOR DE RELATÓRIOS (CLIENTE) =====
public class ReportGenerator
{
    private readonly IReportFactory _factory;

    public ReportGenerator(IReportFactory factory)
    {
        _factory = factory;
    }

    public void GenerateReport(string title, string[][] tableData, double[] chartData)
    {
        var header = _factory.CreateHeader();
        var table = _factory.CreateTable();
        var chart = _factory.CreateChart();

        header.Render(title);
        Console.WriteLine();
        table.Render(tableData);
        Console.WriteLine();
        chart.Render(chartData);
        Console.WriteLine();
    }
```

```
}

// ===== PROGRAMA =====
class Program
{
    static void Main()
    {
        Console.WriteLine("=== EXEMPLO 3: ABSTRACT FACTORY ===\n");

        // Dados de exemplo
        var title = "Relatório de Vendas Q4 2024";
        var tableData = new[]
        {
            new[] { "Produto", "Vendas", "Receita" },
            new[] { "Notebook", "150", "R$ 450.000" },
            new[] { "Mouse", "500", "R$ 25.000" },
            new[] { "Teclado", "300", "R$ 45.000" }
        };
        var chartData = new[] { 450.0, 25.0, 45.0 };

        // Configurar DI
        var services = new ServiceCollection();

        // Registrar componentes PDF
        services.AddTransient<PdfReportHeader>();
        services.AddTransient<PdfReportTable>();
        services.AddTransient<PdfReportChart>();
        services.AddTransient<PdfReportFactory>();

        // Registrar componentes Excel
        services.AddTransient<ExcelReportHeader>();
        services.AddTransient<ExcelReportTable>();
        services.AddTransient<ExcelReportChart>();
        services.AddTransient<ExcelReportFactory>();

        // Registrar componentes HTML
        services.AddTransient<HtmlReportHeader>();
        services.AddTransient<HtmlReportTable>();
        services.AddTransient<HtmlReportChart>();
        services.AddTransient<HtmlReportFactory>();

        var provider = services.BuildServiceProvider();

        // Gerar relatório em PDF
        Console.WriteLine("--- Relatório em PDF ---");
        var pdfFactory = provider.GetRequiredService<PdfReportFactory>();
        var pdfGenerator = new ReportGenerator(pdfFactory);
```

```
        pdfGenerator.GenerateReport(title, tableData, chartData);

        // Gerar relatório em Excel
        Console.WriteLine("--- Relatório em Excel ---");
        var excelFactory = provider.GetRequiredService<ExcelReportFactory>();
        var excelGenerator = new ReportGenerator(excelFactory);
        excelGenerator.GenerateReport(title, tableData, chartData);

        // Gerar relatório em HTML
        Console.WriteLine("--- Relatório em HTML ---");
        var htmlFactory = provider.GetRequiredService<HtmlReportFactory>();
        var htmlGenerator = new ReportGenerator(htmlFactory);
        htmlGenerator.GenerateReport(title, tableData, chartData);

        Console.WriteLine("=== FIM DO EXEMPLO 3 ===");
    }
}
```

EXEMPLO 4: INTEGRAÇÃO AVANÇADA COM DI (50 minutos)

Objetivo

Demonstrar padrões avançados: Func<T>, Named Factory, Generic Factory.

PaymentSystem.csproj

xml

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.DependencyInjection" Version="8.0.0" />
    <PackageReference Include="Microsoft.Extensions.Logging.Console" Version="8.0.0" />
  </ItemGroup>
</Project>
```

Program.cs

csharp

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

// ===== MODELOS =====
public record Payment(string Type, decimal Amount, string Customer);
public record PaymentResult(bool Success, string Message, string TransactionId);

// ===== PROCESSADORES =====
public interface IPaymentProcessor
{
    PaymentResult Process(Payment payment);
}

public class CreditCardProcessor : IPaymentProcessor
{
    private readonly ILogger<CreditCardProcessor> _logger;

    public CreditCardProcessor(ILogger<CreditCardProcessor> logger)
    {
        _logger = logger;
    }

    public PaymentResult Process(Payment payment)
    {
        _logger.LogInformation($"Processando cartão de crédito: {payment.Amount:C}");
        var txId = Guid.NewGuid().ToString()[..8];
        return new PaymentResult(true, "Aprovado via cartão", txId);
    }
}

public class BoletoProcessor : IPaymentProcessor
{
    private readonly ILogger<BoletoProcessor> _logger;

    public BoletoProcessor(ILogger<BoletoProcessor> logger)
    {
        _logger = logger;
    }

    public PaymentResult Process(Payment payment)
    {
        _logger.LogInformation($"Gerando boleto: {payment.Amount:C}");
        var txId = Guid.NewGuid().ToString()[..8];
        return new PaymentResult(true, "Boleto gerado", txId);
    }
}
```

```
public class PixProcessor : IPaymentProcessor
{
    private readonly ILogger<PixProcessor> _logger;

    public PixProcessor(ILogger<PixProcessor> logger)
    {
        _logger = logger;
    }

    public PaymentResult Process(Payment payment)
    {
        _logger.LogInformation($"Processando PIX: {payment.Amount:C}");
        var txId = Guid.NewGuid().ToString()[..8];
        return new PaymentResult(true, "PIX aprovado instantaneamente", txId);
    }
}

// ===== PATTERN 1: FUNC<T> FACTORY =====
public class PaymentServiceWithFunc
{
    private readonly Func<string, IPaymentProcessor> _processorFactory;

    public PaymentServiceWithFunc(Func<string, IPaymentProcessor> processorFactory)
    {
        _processorFactory = processorFactory;
    }

    public PaymentResult ProcessPayment(Payment payment)
    {
        var processor = _processorFactory(payment.Type);
        return processor.Process(payment);
    }
}

// ===== PATTERN 2: GENERIC FACTORY =====
public interface IFactory<T>
{
    T Create();
}

public class Factory<T> : IFactory<T> where T : class
{
    private readonly IServiceProvider _serviceProvider;

    public Factory(IServiceProvider serviceProvider)
    {
```

```
        _serviceProvider = serviceProvider;
    }

    public T Create()
    {
        return _serviceProvider.GetRequiredService<T>();
    }
}

// ===== PATTERN 3: NAMED FACTORY =====
public interface INamedFactory<T>
{
    T Create(string name);
    IEnumerable<string> GetAvailableNames();
}

public class PaymentProcessorFactory : INamedFactory<IPaymentProcessor>
{
    private readonly IServiceProvider _serviceProvider;
    private readonly Dictionary<string, Type> _processors;

    public PaymentProcessorFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
        _processors = new Dictionary<string, Type>(StringComparer.OrdinalIgnoreCase)
        {
            ["credit"] = typeof(CreditCardProcessor),
            ["boleto"] = typeof(BoletoProcessor),
            ["pix"] = typeof(PixProcessor)
        };
    }

    public IPaymentProcessor Create(string name)
    {
        if (!_processors.TryGetValue(name, out var type))
        {
            throw new NotSupportedException($"Tipo de pagamento não suportado: {name}");
        }

        return (IPaymentProcessor)_serviceProvider.GetRequiredService(type);
    }

    public IEnumerable<string> GetAvailableNames()
    {
        return _processors.Keys;
    }
}
```

```
// ===== PROGRAMA =====
class Program
{
    static void Main()
    {
        Console.WriteLine("=== EXEMPLO 4: INTEGRAÇÃO AVANÇADA COM DI ===\n");

        // Configurar DI
        var services = new ServiceCollection();
        services.AddLogging(builder => builder.AddConsole());

        // Registrar processadores
        services.AddTransient<CreditCardProcessor>();
        services.AddTransient<BoletoProcessor>();
        services.AddTransient<PixProcessor>();

        // Pattern 1: Func<T> Factory
        services.AddTransient<Func<string, IPaymentProcessor>>(provider =>
        {
            return type => type.ToLower() switch
            {
                "credit" => provider.GetRequiredService<CreditCardProcessor>(),
                "boleto" => provider.GetRequiredService<BoletoProcessor>(),
                "pix" => provider.GetRequiredService<PixProcessor>(),
                _ => throw new NotSupportedException($"Tipo desconhecido: {type}")
            };
        });
        services.AddTransient<PaymentServiceWithFunc>();

        // Pattern 2: Generic Factory
        services.AddTransient(typeof(IFactory<>), typeof(Factory<>));

        // Pattern 3: Named Factory
        services.AddSingleton<INamedFactory<IPaymentProcessor>, PaymentProcessorFactory>();

        var provider = services.BuildServiceProvider();

        // Demonstração Pattern 1: Func<T>
        Console.WriteLine("--- Pattern 1: Func<T> Factory ---");
        var funcService = provider.GetRequiredService<PaymentServiceWithFunc>();
        var result1 = funcService.ProcessPayment(new Payment("credit", 150.00m, "João Silva"));
        Console.WriteLine($"Resultado: {result1.Message} (ID: {result1.TransactionId})\n");

        // Demonstração Pattern 2: Generic Factory
        Console.WriteLine("--- Pattern 2: Generic Factory ---");
        var genericFactory = provider.GetRequiredService<IFactory<PixProcessor>>();
```

```
        var pixProcessor = genericFactory.Create();
        var result2 = pixProcessor.Process(new Payment("pix", 75.50m, "Maria Santos"));
        Console.WriteLine($"Resultado: {result2.Message} (ID: {result2.TransactionId})\n");

        // Demonstração Pattern 3: Named Factory
        Console.WriteLine("--- Pattern 3: Named Factory ---");
        var namedFactory = provider.GetRequiredService<INamedFactory<IPaymentProcessor>>();

        Console.WriteLine($"Tipos disponíveis: {string.Join(", ", namedFactory.GetAvailableNames())}");

        var boletoProcessor = namedFactory.Create("boleto");
        var result3 = boletoProcessor.Process(new Payment("boleto", 200.00m, "Carlos Oliveira"));
        Console.WriteLine($"Resultado: {result3.Message} (ID: {result3.TransactionId})\n");

        Console.WriteLine("=== FIM DO EXEMPLO 4 ===");
    }
}
```

EXEMPLO 5: PROJECT FACTORY - SISTEMA MODULAR (60 minutos)

Este exemplo é o mais complexo e demonstra uma aplicação modular completa com descoberta dinâmica de módulos.

ModularApp.csproj

xml

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.DependencyInjection" Version="8.0.0" />
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="8.0.0" />
    <PackageReference Include="Microsoft.Extensions.Logging.Console" Version="8.0.0" />
  </ItemGroup>
</Project>
```

Program.cs

csharp

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using System.Reflection;

// ===== INFRAESTRUTURA DE MÓDULOS =====

[AttributeUsage(AttributeTargets.Class)]
public class ModuleAttribute : Attribute
{
    public string Name { get; set; } = string.Empty;
    public string[] Dependencies { get; set; } = Array.Empty<string>();
    public bool IsOptional { get; set; } = true;
}

public interface IModule
{
    string Name { get; }
    string Version { get; }
    string Description { get; }
    void ConfigureServices(IServiceCollection services);
    void Initialize(IServiceProvider serviceProvider);
}

public class ModuleInfo
{
    public string Name { get; set; } = string.Empty;
    public string Version { get; set; } = string.Empty;
    public string[] Dependencies { get; set; } = Array.Empty<string>();
    public bool IsOptional { get; set; }
}

// ===== PROJECT FACTORY =====

public interface IProjectFactory
{
    void BuildProject(IServiceCollection services);
    void InitializeModules(IServiceProvider serviceProvider);
}

public class ProjectFactory : IProjectFactory
{
    private readonly ILogger<ProjectFactory> _logger;
    private readonly List<IModule> _modules;

    public ProjectFactory(ILogger<ProjectFactory> logger)
```

```
{
    _logger = logger;
    _modules = new List<IModule>();
}

public void BuildProject(IServiceCollection services)
{
    _logger.LogInformation("=== Iniciando construção do projeto ===");

    // Descobrir módulos no assembly atual
    var moduleTypes = Assembly.GetExecutingAssembly()
        .GetTypes()
        .Where(t => typeof(IModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);

    var discoveredModules = moduleTypes
        .Select(t => (IModule)Activator.CreateInstance(t)!)
        .ToList();

    _logger.LogInformation($"Descobertos {discoveredModules.Count} módulos");

    // Ordenar por dependências
    var orderedModules = OrderByDependencies(discoveredModules);

    // Carregar cada módulo
    foreach (var module in orderedModules)
    {
        try
        {
            _logger.LogInformation($"Carregando módulo: {module.Name} v{module.Version}");
            module.ConfigureServices(services);
            _modules.Add(module);
            _logger.LogInformation($"✓ Módulo {module.Name} carregado");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"✗ Erro ao carregar módulo {module.Name}");

            var attr = module.GetType().GetCustomAttribute<ModuleAttribute>();
            if (attr?.IsOptional == false)
            {
                throw;
            }
        }
    }

    _logger.LogInformation($"=== Projeto construído com {_modules.Count} módulos ===\n");
}
```

```
public void InitializeModules(IServiceProvider serviceProvider)
{
    _logger.LogInformation("=== Inicializando módulos ===");

    foreach (var module in _modules)
    {
        _logger.LogInformation($"Inicializando: {module.Name}");
        module.Initialize(serviceProvider);
    }

    _logger.LogInformation("=== Todos os módulos inicializados ===\n");
}

private List<IModule> OrderByDependencies(List<IModule> modules)
{
    var sorted = new List<IModule>();
    var visited = new HashSet<string>();

    void Visit(IModule module)
    {
        if (visited.Contains(module.Name)) return;

        visited.Add(module.Name);

        var attr = module.GetType().GetCustomAttribute<ModuleAttribute>();
        if (attr?.Dependencies != null)
        {
            foreach (var depName in attr.Dependencies)
            {
                var dependency = modules.FirstOrDefault(m => m.Name == depName);
                if (dependency != null)
                {
                    Visit(dependency);
                }
            }
        }

        sorted.Add(module);
    }

    foreach (var module in modules)
    {
        Visit(module);
    }

    return sorted;
```

```
    }
}

// ===== MÓDULOS CONCRETOS =====

// Módulo base - sem dependências
[Module(Name = "Logging", IsOptional = false)]
public class LoggingModule : IModule
{
    public string Name => "Logging";
    public string Version => "1.0.0";
    public string Description => "Módulo de logging centralizado";

    public void ConfigureServices(IServiceCollection services)
    {
        // Logging já configurado pelo Host, apenas exemplo
        services.AddSingleton<ILoggerService, LoggerService>();
    }

    public void Initialize(IServiceProvider serviceProvider)
    {
        var logger = serviceProvider.GetRequiredService<ILoggerService>();
        logger.Log("Módulo de Logging inicializado");
    }
}

public interface ILoggerService
{
    void Log(string message);
}

public class LoggerService : ILoggerService
{
    private readonly ILogger<LoggerService> _logger;

    public LoggerService(ILogger<LoggerService> logger)
    {
        _logger = logger;
    }

    public void Log(string message)
    {
        _logger.LogInformation($"[APP] {message}");
    }
}

// Módulo de notificações - depende de Logging
```

```
[Module(Name = "Notifications", Dependencies = new[] { "Logging" }, IsOptional = false)]
public class NotificationsModule : IModule
{
    public string Name => "Notifications";
    public string Version => "1.0.0";
    public string Description => "Sistema de notificações multi-canal";

    public void ConfigureServices(IServiceCollection services)
    {
        services.AddTransient<INotificationService, NotificationService>();
    }

    public void Initialize(IServiceProvider serviceProvider)
    {
        var logger = serviceProvider.GetRequiredService<ILoggerService>();
        logger.Log("Módulo de Notificações inicializado");
    }
}

public interface INotificationService
{
    void Notify(string message);
}

public class NotificationService : INotificationService
{
    private readonly ILoggerService _logger;

    public NotificationService(ILoggerService logger)
    {
        _logger = logger;
    }

    public void Notify(string message)
    {
        _logger.Log($"📢 Notificação: {message}");
    }
}

// Módulo de pagamentos - depende de Logging e Notifications
[Module(Name = "Payments", Dependencies = new[] { "Logging", "Notifications" }, IsOptional = false)]
public class PaymentsModule : IModule
{
    public string Name => "Payments";
    public string Version => "1.0.0";
    public string Description => "Sistema de processamento de pagamentos";
```

```
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddTransient<IPaymentService, PaymentService>();
    }

    public void Initialize(IServiceProvider serviceProvider)
    {
        var logger = serviceProvider.GetRequiredService<ILoggerService>();
        logger.Log("Módulo de Pagamentos inicializado");
    }
}

public interface IPaymentService
{
    void ProcessPayment(decimal amount);
}

public class PaymentService : IPaymentService
{
    private readonly ILoggerService _logger;
    private readonly INotificationService _notifier;

    public PaymentService(ILoggerService logger, INotificationService notifier)
    {
        _logger = logger;
        _notifier = notifier;
    }

    public void ProcessPayment(decimal amount)
    {
        _logger.Log($"💳 Processando pagamento de {amount:C}");
        _notifier.Notify($"Pagamento de {amount:C} processado com sucesso");
    }
}

// ===== APLICAÇÃO PRINCIPAL =====

public class Application : IHostedService
{
    private readonly IPaymentService _paymentService;
    private readonly ILoggerService _logger;

    public Application(IPaymentService paymentService, ILoggerService logger)
    {
        _paymentService = paymentService;
        _logger = logger;
    }
```

```
    public Task StartAsync(CancellationToken cancellationToken)
    {
        _logger.Log("=== Aplicação iniciada ===");

        // Simular operações
        _paymentService.ProcessPayment(150.00m);
        _paymentService.ProcessPayment(75.50m);

        _logger.Log("=== Demonstração concluída ===");

        return Task.CompletedTask;
    }

    public Task StopAsync(CancellationToken cancellationToken)
    {
        return Task.CompletedTask;
    }
}

// ===== PROGRAMA =====

class Program
{
    static async Task Main(string[] args)
    {
        Console.WriteLine("=== EXEMPLO 5: PROJECT FACTORY - SISTEMA MODULAR ===\n");

        var host = Host.CreateDefaultBuilder(args)
            .ConfigureServices((context, services) =>
            {
                // Registrar infraestrutura
                services.AddSingleton<IProjectFactory, ProjectFactory>();

                // Construir projeto (carregar módulos)
                var tempProvider = services.BuildServiceProvider();
                var factory = tempProvider.GetRequiredService<IProjectFactory>();
                factory.BuildProject(services);

                // Registrar aplicação principal
                services.AddHostedService<Application>();
            })
            .Build();

        // Inicializar módulos
        var factory = host.Services.GetRequiredService<IProjectFactory>();
        factory.InitializeModules(host.Services);
```

```
        // Executar aplicação
        await host.RunAsync();
    }
}
```

INSTRUÇÕES DE COMPILAÇÃO E EXECUÇÃO

Criar e executar cada exemplo:

```
bash

# Exemplo 1
cd 01-simple-factory
dotnet run

# Exemplo 2
cd ../02-factory-method
dotnet run

# Exemplo 3
cd ../03-abstract-factory
dotnet run

# Exemplo 4
cd ../04-di-integration
dotnet run

# Exemplo 5
cd ../05-project-factory
dotnet run
```

Criar solução completa:

```
bash

dotnet new sln -n FactoryPatterns
dotnet sln add 01-simple-factory/SimpleFactory.csproj
dotnet sln add 02-factory-method/NotificationService.csproj
dotnet sln add 03-abstract-factory/ReportSystem.csproj
dotnet sln add 04-di-integration/PaymentSystem.csproj
dotnet sln add 05-project-factory/ModularApp.csproj
```

NOTAS PEDAGÓGICAS

1. **Progressão didática**: Os exemplos aumentam em complexidade gradualmente
2. **Código comentado**: Comentários explicam decisões de design importantes
3. **Exemplos executáveis**: Todos os projetos podem ser executados imediatamente
4. **Três domínios**: Notificações, Relatórios e Pagamentos (conforme solicitado)
5. **Integração com DI**: Todos os exemplos demonstram uso correto de DI
6. **Boas práticas**: Código segue princípios SOLID e padrões .NET